

Software Requirements Specification (SRS)

Project Title: Library Management System (LMS)

Project Type: Full-stack Web Development Capstone Project

Document Version: 1.0

Author: Kelvin Wong

Date: 02 December 2025

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) precisely defines all functional and non-functional requirements for the Library Management System (LMS), a full-stack web application developed using Spring Boot (Backend), React.js (Frontend), and MySQL (Database).

1.2 Document Conventions

- **Shall** = Mandatory
- **Should** = Strongly recommended
- **May** = Optional

1.3 Intended Audience

- Capstone instructors and examiners
- Developers and future maintainers
- Project reviewers

1.4 Project Scope

Refer to approved Business Requirements Document.

2. Overall Description

2.1 Product Perspective

The LMS is a standalone digital solution replacing manual library processes with automated member management, book cataloguing, and circulation control.

2.2 Product Functions Summary

- Secure authentication & role-based access
- Member CRUD (Librarian only)
- Book catalogue management with search/filter
- Full borrowing, return, renewal, and reservation workflow
- Automatic fine calculation
- Member self-service portal

2.3 User Classes and Characteristics

User Class	Frequency of Use	Technical Proficiency	Primary Activities
Librarian (Admin)	Daily	Moderate to High	All system functions
Member	Several times/week	Low to Moderate	Search, reserve, view history

2.4 Operating Environment

- Client: Modern browsers (latest two versions of Chrome, Firefox, Edge, Safari)
- Server: Java 17+, MySQL 8+, any cloud or on-premise host

2.5 Design & Implementation Constraints

- Backend: Spring Boot 3.x, Java 17, Maven/Gradle, Spring Security + JWT
- Frontend: React 18+, React Router v6, Material-UI or Bootstrap 5, Axios
- Database: MySQL 8.0+

2.6 Assumptions & Dependencies

- Stable internet connection for end users
- Email server available for password reset (can be console-logged during dev)

3. System Features

3.1 Authentication & Authorization

Priority: Critical

ID	Requirement
FRS-AUTH-01	System shall allow self-registration for Members
FRS-AUTH-02	System shall support login using email and password
FRS-AUTH-03	System shall enforce role-based access (Librarian/Admin vs Member)
FRS-AUTH-04	System shall provide “Forgot Password” with secure reset token via email

3.2 Member Management (Admin only)

Priority: High

ID	Requirement
FRS-MEM-01	Librarian shall perform CRUD on member records
FRS-MEM-02	Member fields: MemberID (auto), Full Name, Email, Phone, Address, Registration Date, Expiry (1 year)
FRS-MEM-03	Librarian shall search members by name, MemberID

3.3 Book Management

Priority: High

ID	Requirement
FRS-BOOK-01	Librarian shall add, edit, delete, and view books
FRS-BOOK-02	Book fields: ISBN (unique), Title, Author(s), Category, Publisher, Year, Total Copies, Available Copies
FRS-BOOK-03	Users shall search/filter by Title, Author, ISBN, Category, Year
FRS-BOOK-04	System shall show real-time status: Available / Borrowed / Reserved

3.4 Lending Management

Priority: Critical

ID	Requirement	Business Rule
FRS-CIRC-01	Borrow only if: active membership, < 3 books out, no overdue, fines $\leq$ \$10	Auto validation
FRS-CIRC-02	Default loan period = 14 days	—
FRS-CIRC-03	Member may renew max 2 times if no pending reservation	Max 2 renewals
FRS-CIRC-04	System shall maintain reservation queue (FIFO)	—
FRS-CIRC-05	On return: auto-calculate overdue fine (\$0.50/day, capped at \$20 per book)	Auto calculation
FRS-CIRC-06	Member shall view own current loans, history, reservations, and total fine	—
FRS-CIRC-07	Librarian shall view all loans, overdue list, and reservation queue	—

4. External Interface Requirements

4.1 User Interfaces

- Responsive, clean UI using Material-UI or Bootstrap
- Role-based navigation menu
- Toast/snackbar for success & error messages

4.2 REST API Endpoints (examples)

- POST /api/auth/login
- GET /api/books?search=&category=
- POST /api/transactions/borrow (Full API spec to be provided separately)

5. Non-Functional Requirements (Good to have)

ID	Category	Requirement	Verification Method
NFR-01	Performance	Page load $\leq$ 2 sec under 100 concurrent users	JMeter / k6
NFR-02	Security	BCrypt hashing, JWT (24h expiry), HTTPS only	OWASP scan
NFR-03	Reliability	Borrow/return transactions atomic	@Transactional tests
NFR-04	Scalability	Support 10,000 books & 5,000 members	Load testing
NFR-05	Usability	New users complete core tasks in $<$ 5 min without help	Usability testing
NFR-06	Maintainability	Backend test coverage $\geq$ 70%	JaCoCo report
NFR-07	Compatibility	Fully responsive $\geq$ 768px, latest 4 major browsers	BrowserStack

6. Other Requirements

- English only (v1.0)
- All borrow/return events logged
- Database backup strategy documented separately